# Modulo 4

## Cloud Architecting y Alta Disponibilidad

Curso: AWS Cloud Practitioner Essentials

6 horas

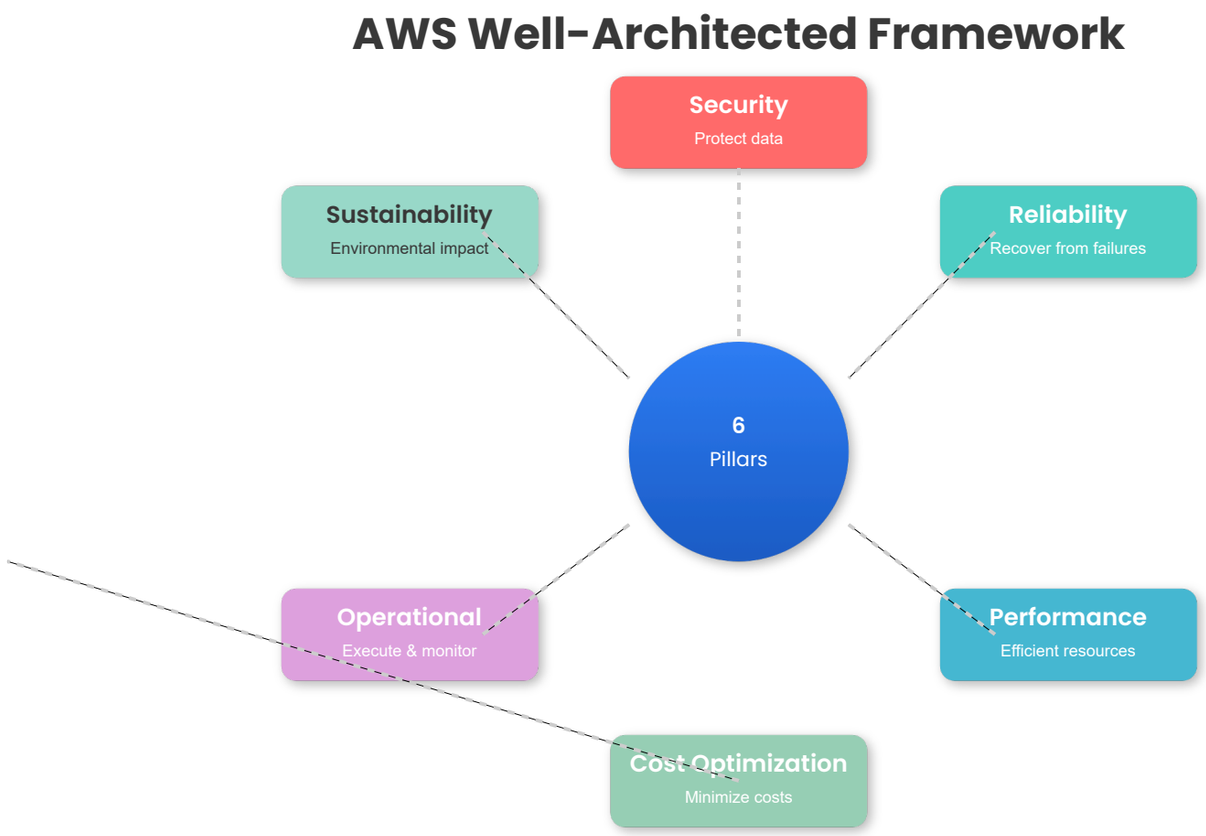
# Objetivos de Aprendizaje

- Explicar los seis pilares del AWS Well-Architected Framework
- Aplicar los principios de diseno para arquitecturas en la cloud
- Disenar arquitecturas de alta disponibilidad (multi-AZ) escalables
- Implementar balanceadores de carga (ALB/NLB) para distribucion de trafico
- Configurar Amazon Route 53 con politicas de routing apropiadas
- Desplegar arquitecturas de 3 capas (web, aplicacion, datos)
- Implementar caching strategies con ElastiCache y CloudFront
- Describir patrones de mensajeria asincrona con SQS y SNS
- Crear Functions Lambda para procesamiento de eventos
- Implementar Infrastructure as Code basico con CloudFormation

# 1. AWS Well-Architected Framework

Mejores practicas para diseno de arquitecturas

# Los 6 Pilares del Framework



# Pilar: Operational Excellence

**Focus:** Ejecutar y monitorear sistemas para entregar valor de negocio

- Performs operations as code
- Make frequent, small, reversible changes
- Refine operations procedures frequently
- Anticipate failure
- Learn from all operational failures

# Pilar: Security

**Focus:** Proteger informacion y sistemas

- Implement strong identity foundation
- Enable traceability
- Apply security at all layers
- Protect data in transit and at rest
- Prepare for security events

# Pilar: Reliability

**Focus:** Asegurar que el sistema se recupera de fallos

- Automatically recover from failure
- Test recovery procedures
- Scale horizontally for resilience
- Manage change in automation

# Pilar: Performance Efficiency

- Democratize advanced technologies
- Go global in minutes
- Use serverless architectures
- Experiment more often

# Pilar: Cost Optimization

- Implement Cloud Financial Management
- Adopt consumption models
- Measure overall efficiency
- Stop spending money on undifferentiated heavy lifting

# Pilar: Sustainability

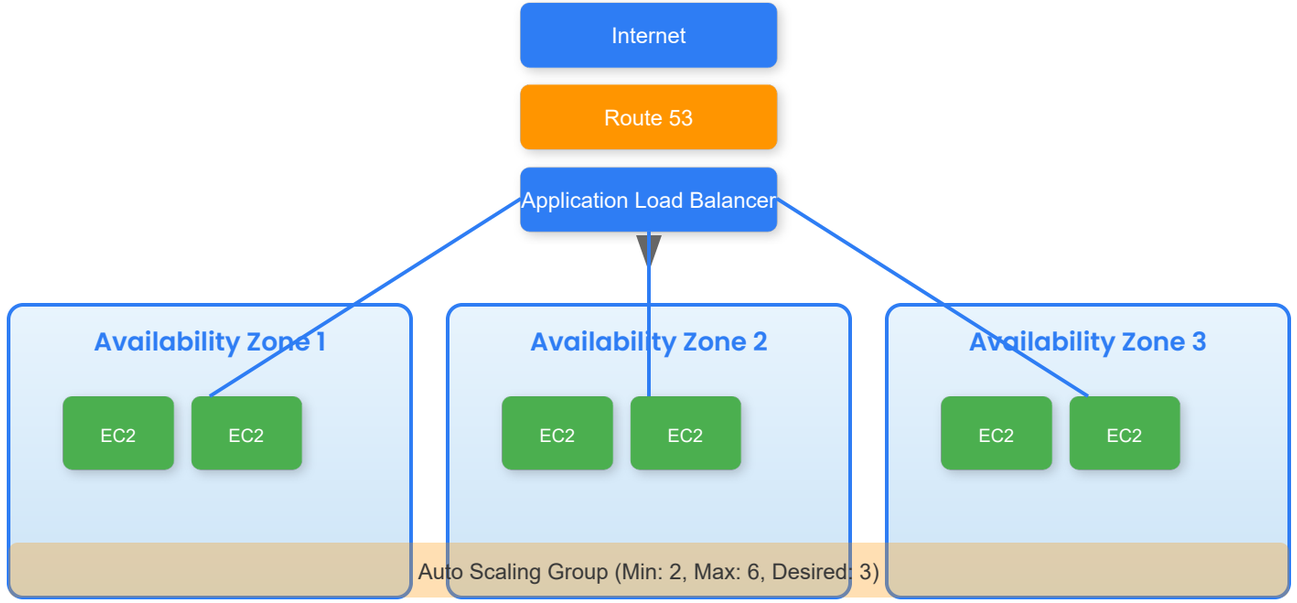
- Understand your impact
- Maximize utilization
- Use managed services
- Reduce downstream impact

# 2. Alta Disponibilidad y Escalabilidad

Multi-AZ, Load Balancing, Auto Scaling

# Arquitectura Multi-AZ vs Multi-Region

## Arquitectura Multi-AZ con Auto Scaling



# Comparativa Multi-AZ vs Multi-Region

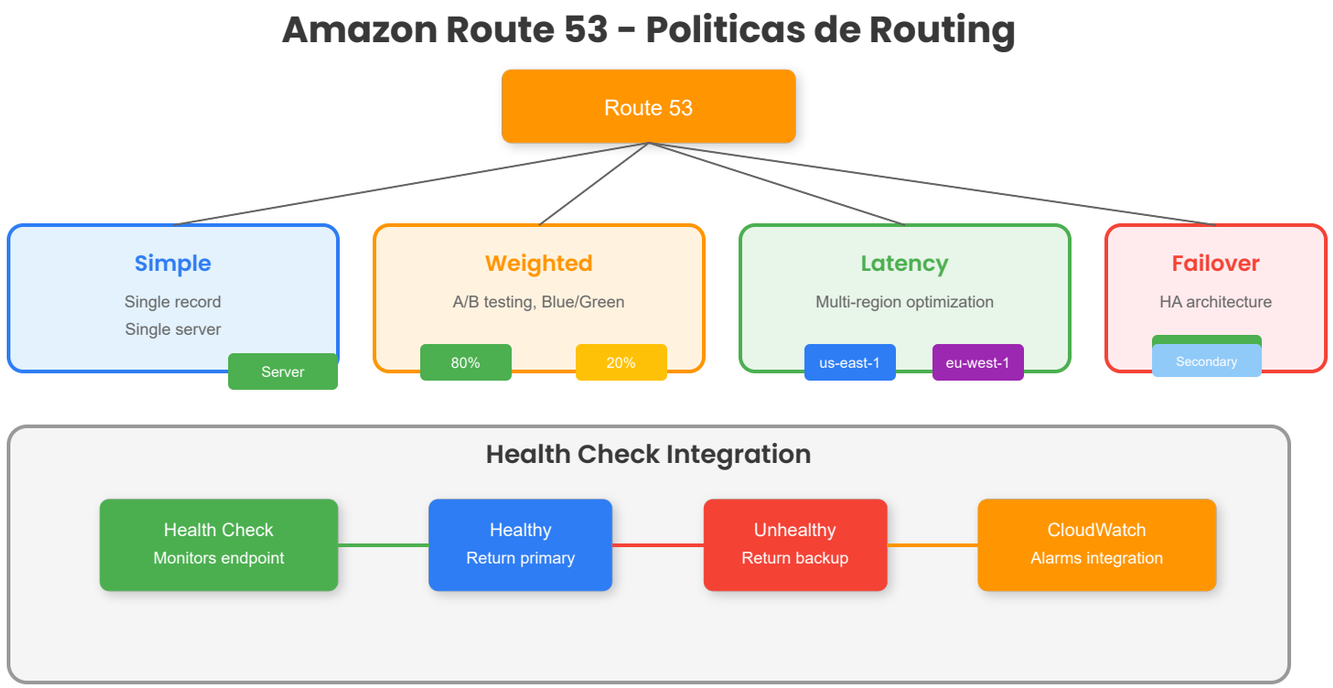
| Aspecto | Multi-AZ | Multi-Region |
|---|---|---|
| Proposito | Alta disponibilidad local | Disaster recovery |
| Latencia | <1ms entre AZs | 100-200ms entre regiones |
| Costo | Moderado | Alto |
| Casos uso | App tier, databases | Backup, DR, global apps |
| RPO | Segundos-minutos | Horas |

# Tipos de Load Balancers

| Tipo | Protocolos | Caso de uso |
|---|---|---|
| ALB | HTTP/HTTPS | Web apps, microservices |
| NLB | TCP/UDP | High performance, low latency |
| CLB | HTTP/TCP | Legacy, migration |

**ALB Features:** Content-based routing, path-based routing, SSL termination, integration with ACM

# Amazon Route 53 - Politicas de Routing



# Auto Scaling Groups

**Componentes:**

- Launch Template: AMI, instance type, SG, userdata
- VPC/Subnets: Multi-AZ distribution
- Load Balancer: ALB target group

**Tipos de Scaling:**

| Tipo | Descripcion |
|---|---|
| Dynamic | Responde a metricas (CPU, Memory) |
| Scheduled | Basado en tiempo (lunes 9am) |
| Predictive | ML-based, anticipate demand |

# Amazon RDS Multi-AZ

**Principales caracteristicas:**

- Replica sincronica a standby en otra AZ
- Automatic failover en ~60 segundos
- Read Replicas para escalabilidad de lectura

| Opcion | Descripcion | RPO |
|---|---|---|
| Single-AZ | Una instancia | N/A |
| Multi-AZ | Sincronous replication | ~0 (seconds) |
| Read Replica | Async replication, read scale | Minutes |

# Lab 4.1: Arquitectura Web de 3 Capas

90 minutos

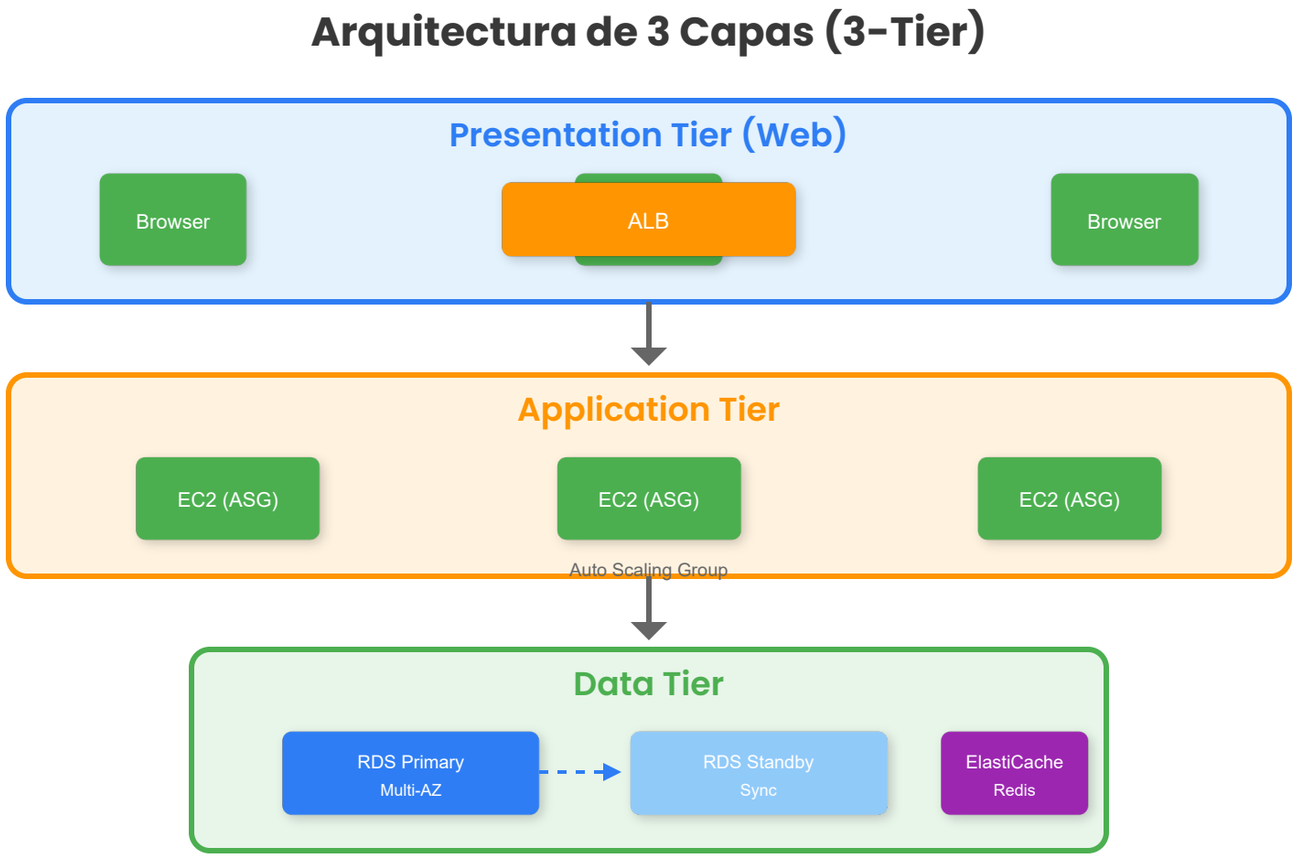
# Pasos del Lab

1. Crear ALB con target groups
2. Crear Launch Template para Web Tier
3. Crear Auto Scaling Group con politicas
4. Crear instancia de aplicacion
5. Crear RDS Multi-AZ
6. Configurar Health Checks y Failover
7. Verificar Alta Disponibilidad

# 3. Patrones de Arquitectura

3-Tier, Caching, CDN, Mensajeria

# Arquitectura de 3 Capas (3-Tier)



# Caching Strategies

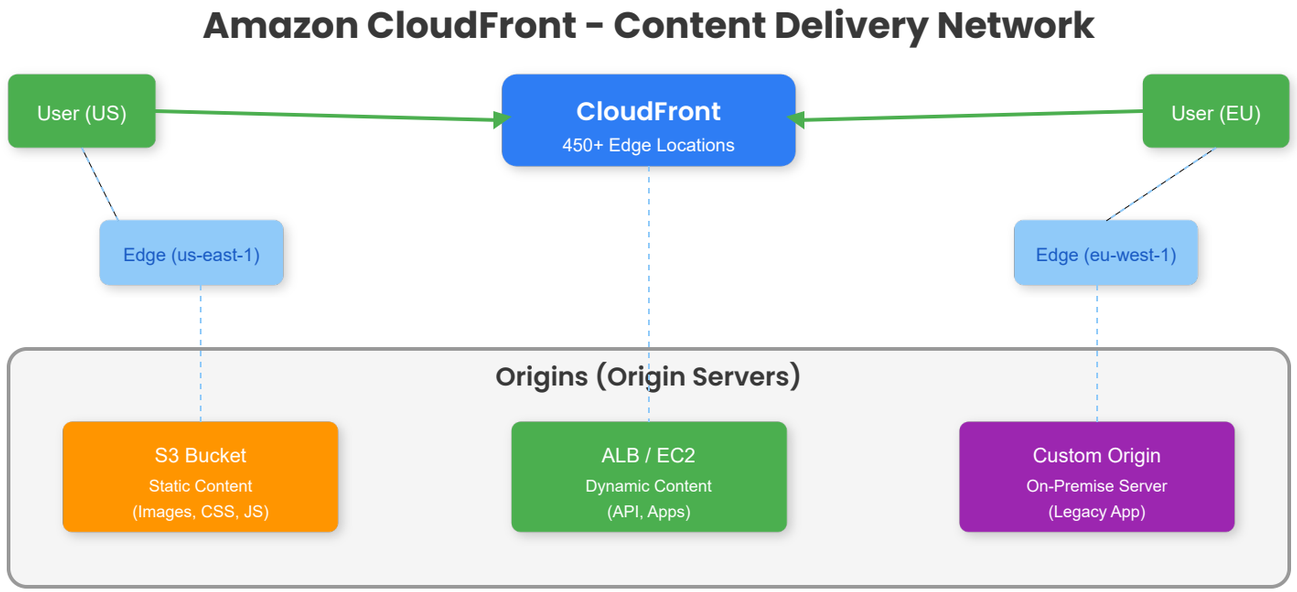
**Cuando usar caching:**

- Datos frecuentemente accedidos
- Datos que cambian raramente
- Latencia critica
- Reduccion de costos de compute

**Amazon ElastiCache:**

- **Redis:** Estructuras complejas, pub/sub, high availability
- **Memcached:** Simple key-value, multi-threaded

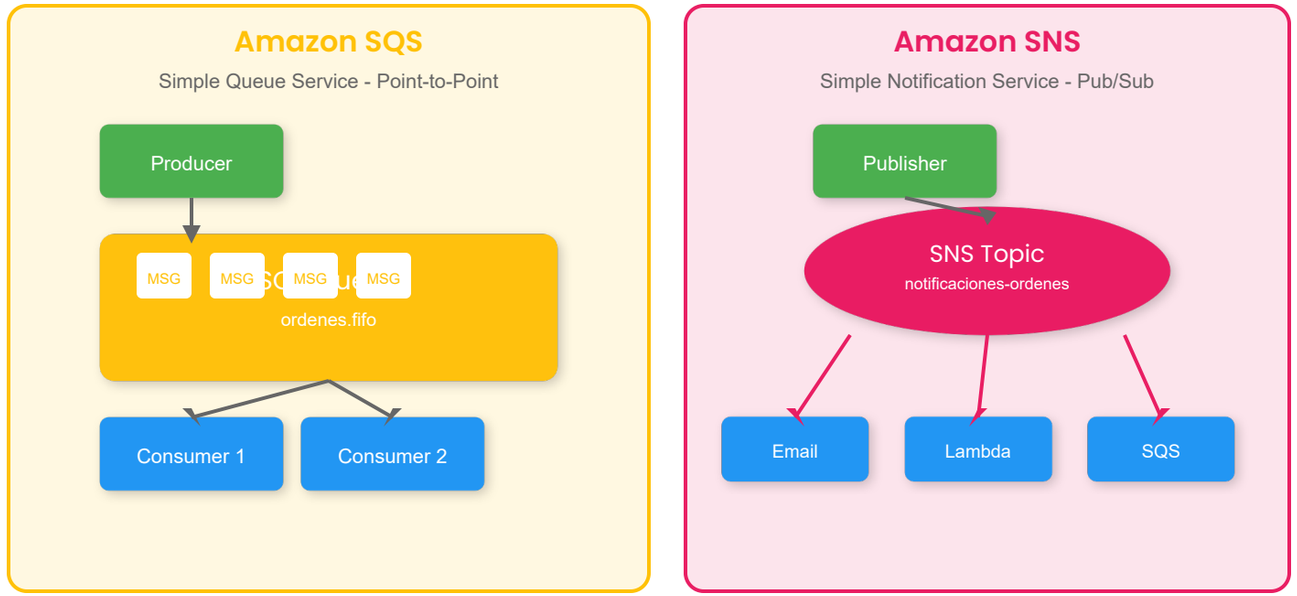
# Amazon CloudFront (CDN)




Cache hit: serve from edge | Cache miss: fetch from origin

# Amazon SQS y SNS

## Amazon SQS y SNS: Mensajeria Asincrona




SQS: Queue | Point-to-Point | Polling

SNS: Topic | Pub/Sub | Push

# SQS vs SNS

| Aspecto | SQS | SNS |
|---|---|---|
| Pattern | Point-to-point (queue) | Pub/Sub (topic) |
| Consumers | Single consumer | Multiple subscribers |
| Delivery | Polling model | Push model |
| Use case | Task queue, async jobs | Notifications, fan-out |

# Lab 4.2: Route 53 con Routing Policies

60 minutos

# Pasos del Lab

1. Crear Hosted Zone en Route 53
2. Crear Health Check
3. Configurar Weighted Routing (A/B Testing)
4. Configurar Latency-Based Routing
5. Configurar Failover Routing
6. Configurar Geolocation Routing
7. Verificar configuracion

# 4. Microservices y Arquitectura Orientada a Eventos

API Gateway, Event-Driven Architecture

# Monolito vs Microservices

## Monolito

```
Single deploy
- Web App
- API
- Database
```

## Microservices

```
Independent deploys
- Service 1
- Service 2
- Service 3
- API Gateway
```

# Amazon API Gateway

**Que es:** Servicio fully managed para crear, publicar y mantener APIs

- Tipos: REST, WebSocket, HTTP APIs
- TLS termination
- Authorization (IAM, Cognito, Lambda)
- Rate limiting
- API versioning

# Arquitectura Orientada a Eventos (EDA)

**Concepto:** Productores y consumidores desacoplados via un Event Router

- **Event Producer:** EC2, Lambda
- **Event Router:** SNS, EventBridge
- **Event Consumer:** Lambda, EC2, SQS

# Amazon EventBridge

- Serverless event bus
- Schema registry
- Integration with SaaS apps
- Event replay

**Patrones EDA:**

- Event sourcing: Guardar secuencia de eventos
- CQRS: Separate read/write models
- Saga: Coordinar transacciones distribuidas

# Lab 4.4: Mensajeria Asincrona con SQS y SNS

45 minutos

# Pasos del Lab

1. Crear SQS Queue (FIFO) con deduplicacion
2. Crear SNS Topic con suscripciones email
3. Crear Lambda function para procesar mensajes
4. Configurar Lambda como SQS Trigger
5. Configurar SNS como DLQ para Lambda
6. Enviar mensajes de prueba
7. Monitorear en CloudWatch

# 5. Infrastructure as Code con CloudFormation

Introduccion a IaC con AWS

# Que es CloudFormation?

## Infraestructura como Codigo (IaC)

- Templates JSON/YAML
- Declarative, no imperative
- Stack: AWS resources creados desde template

## Beneficios:

- Tracks state
- Drift detection
- Rollback on failure

# Estructura de Template

```yaml
AWSTemplateFormatVersion: '2010-09-09'
Description: Mi primera VPC

Parameters:
  EnvironmentName:
    Type: String
    Default: Production

Resources:
  MyVPC:
    Type: AWS::EC2::VPC
    Properties:
      CidrBlock: 10.0.0.0/16
      Tags:
        - Key: Name
          Value: !Ref EnvironmentName

Outputs:
  VPCId:
```

# Comandos Utiles CloudFormation

```
# Crear stack
aws cloudformation create-stack \
  --stack-name mi-vpc \
  --template-body file://vpc.yaml

# Describir stacks
aws cloudformation describe-stacks \
  --stack-name mi-vpc

# Actualizar stack
aws cloudformation update-stack \
  --stack-name mi-vpc \
  --template-body file://vpc.yaml

# Eliminar stack
aws cloudformation delete-stack \
  --stack-name mi-vpc
```

# Lab 4.3: Crear VPC con CloudFormation (Opcional)

45 minutos

# Pasos del Lab

1. Crear template YAML con recursos de VPC, subnets, IGW, NAT
2. Usar aws cloudformation create-stack
3. Verificar recursos creados en consola
4. Modificar template y hacer update-stack

# Resumen

# Puntos Clave

- **Well-Architected:** 6 pilares: Security, Reliability, Performance, Cost, Operational, Sustainability
- **HA Architecture:** Multi-AZ, Load Balancing, Auto Scaling
- **Route 53:** Politicas: Simple, Weighted, Latency, Failover, Geolocation
- **3-Tier Architecture:** Presentation, Application, Data tiers
- **Caching:** ElastiCache (Redis/Memcached), CloudFront
- **Messaging:** SQS (queues), SNS (pub/sub)
- **Microservices:** API Gateway, EventBridge, decoupling
- **CloudFormation:** IaC con templates YAML/JSON

# Referencias

# Recursos Adicionales

- Well-Architected Framework
- AWS Architecture Center
- ALB Documentation
- Route 53 Documentation
- SNS Documentation
- SQS Documentation
- AWS Skill Builder

# Gracias!

## Modulo 4 Completado

Siguiente: Modulo 5 - Redes y Content Delivery